

Lecture 15: The single cycle processor

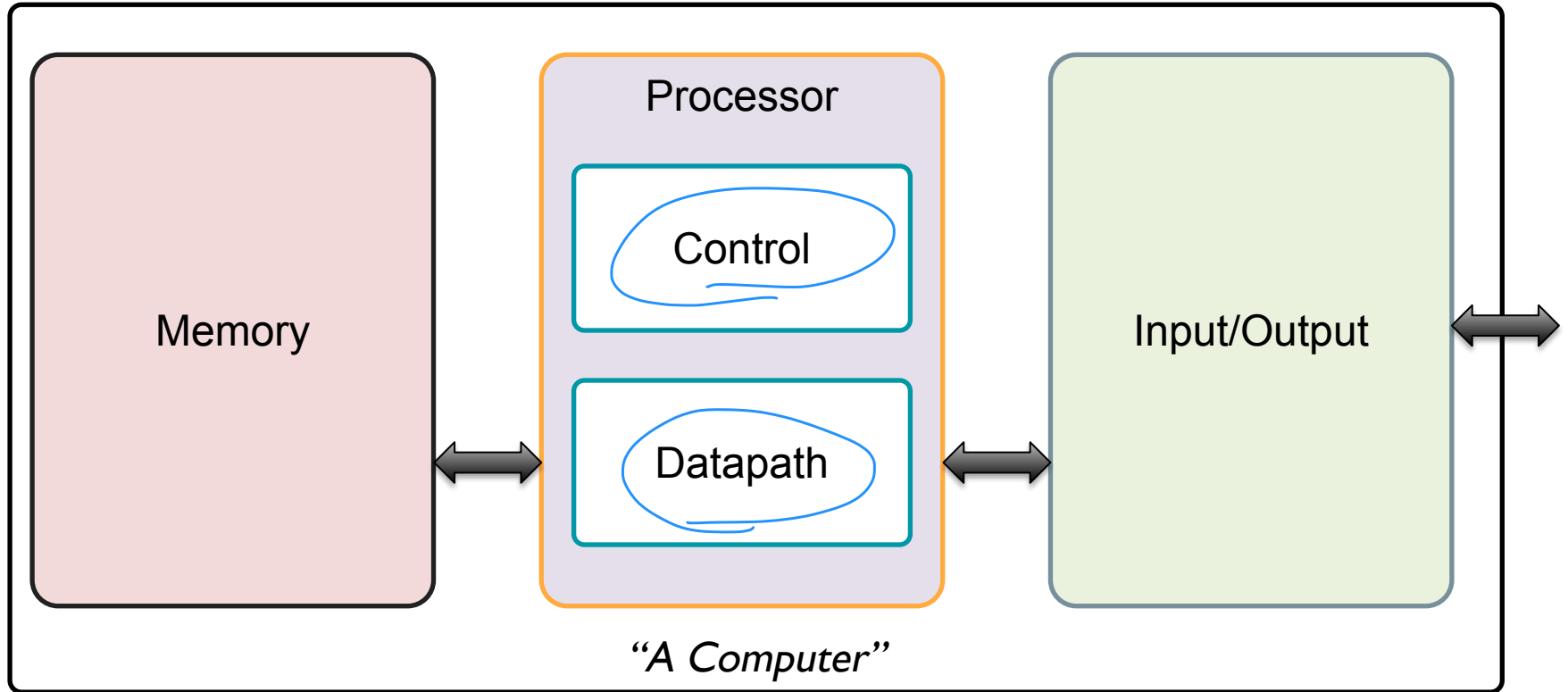
CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- Reminder: check-in with professor is 1% of your final grade
 - Sign up [link](#)
- Midterm Exam in-class next Tuesday, October 13th
 - Can bring 1 double-sided 8.5x11" note sheet (handwritten notes highly recommended!)
 - Cannot bring any other aids (e.g., no calculators, extra notes, etc)
- HW3 is due October 15th, 11:59pm

The major building blocks of a computer



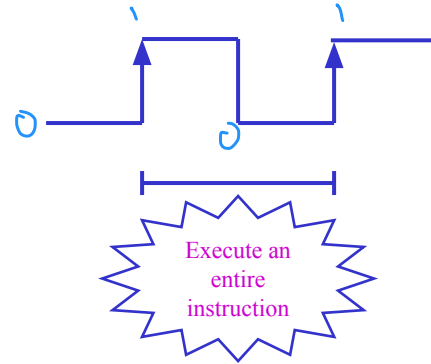
Performance perspective

- Processor design (datapath and control) will determine:
 - Cycle time (CT)
 - Cycles per instruction (CPI)
- Single cycle processor:
 - Advantage: One clock cycle per instruction
 - Disadvantage: long cycle time

→ execution time

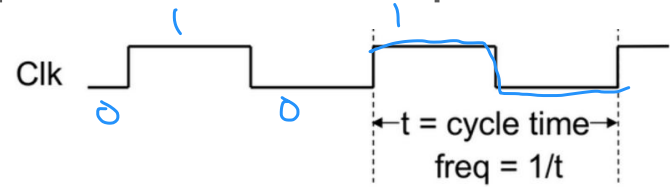
- $ET = \text{Instr Count} * \text{CPI} * \text{CT}$

→ Performance equation



Logic design basics

- From basics we can understand how to implement an ISA as a processor



- **Clock**: measures speed of processor
- **Logic elements**
 - **Combinational** logic
 - **State** (sequential) elements
- **Memory** elements
 - Large storage separate from logic elements

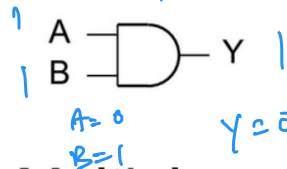
$$\begin{aligned} t &= 1 \text{ ns} \\ \text{freq} &= \frac{1}{t} \\ &= \frac{1}{1 \times 10^{-9} \text{ s}} \\ &= 10^9 \\ &= 1 \text{ GHz} \end{aligned}$$

Combinational logic

- Calculates the output of a **logic function** based on its inputs

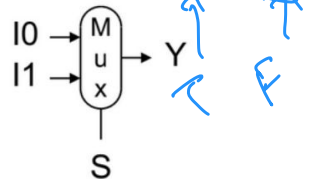
■ AND-gate

■ $Y = A \& B$



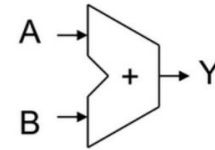
■ Multiplexer (Mux)

■ $Y = S ? I1 : I0$



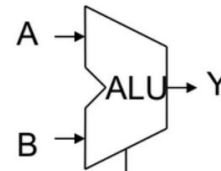
■ Adder

■ $Y = A + B$



■ Arithmetic/Logic Unit

■ $Y = F(A, B)$



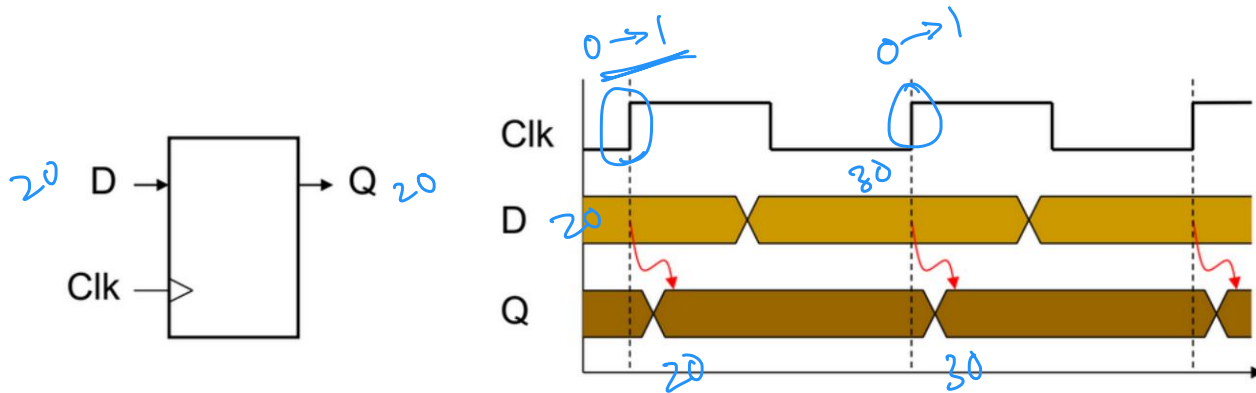
$F = \{+, -, \times, \div, \dots\}$

State (sequential) elements

- Register: "remembers" data in a circuit

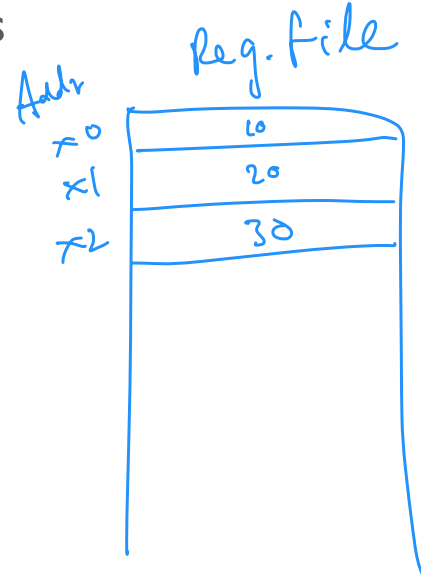
State (sequential) elements

- Register: "remembers" data in a circuit
 - Uses **clock signal** to know when to update the stored value
 - Edge-triggered: update when clk changes 0 → 1
 - Called a **flip-flop**



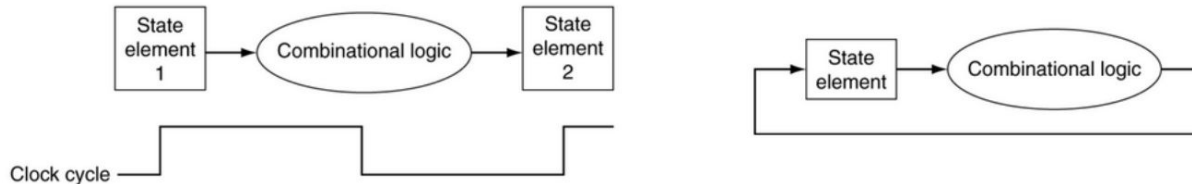
State (sequential) elements

- **Register file:** array of addressable registers
 - Name of register = address
 - Ex: x1 is register address 1



Critical path

- Combinational logic **computes** and **transforms data**
 - Between clock edges
 - Input from state element, output to state element
 - Longest delay determines cycle time
- **Critical path:** Longest combinational logic delay



Memory elements

- **Load**: given address, output data
 - often to a sequential element, e.g., register
- **Store**: given address and data, write data into memory
- Synchronous with clock
 - Issue load/store in cycle i , data is read/written in cycle $i+1$

How does a machine process instructions?

- What does it mean to process an instruction?

How does a machine process instructions?

- What does it mean to **process an instruction**?

AS = Architectural
(programmer visible) state
BEFORE processing instr

Reg PC Mem

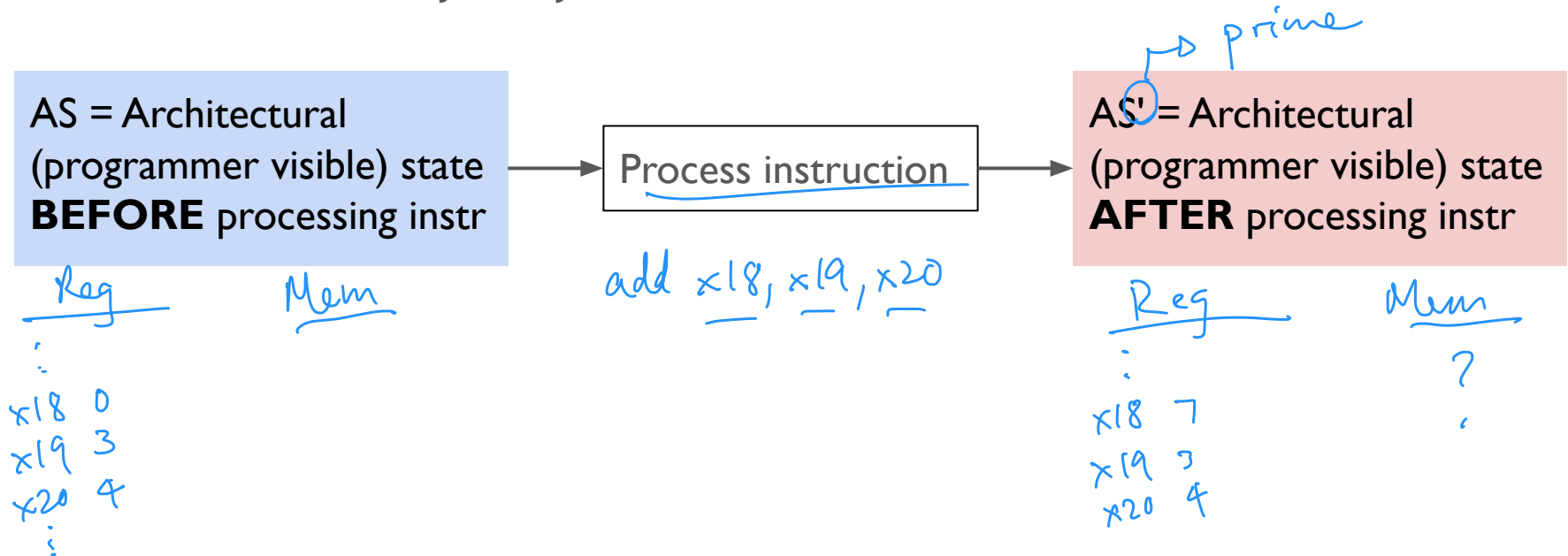
How does a machine process instructions?

- What does it mean to **process an instruction**?



How does a machine process instructions?

- What does it mean to **process an instruction**?
 - Transform **AS** to **AS'** based on the ISA instruction specification
 - Ex: add x18, x19, x20



Programmer visible (architectural) state

- Memory
- Registers
- Program counter (PC)
 - register storing memory address of current instruction

Programmer visible (architectural) state

- Memory

Instructions (and programs) specify how to transform values of programmer visible state

- Registers

- Program counter (PC)

- register storing memory address of current instruction

Instruction processing

- ISA specifies what AS' should be given instruction and AS

Instruction processing

- ISA specifies what AS' should be given instruction and AS
- Microarchitecture implements how AS transforms to AS'
 - How instruction is processed by machine (organization)

Instruction processing

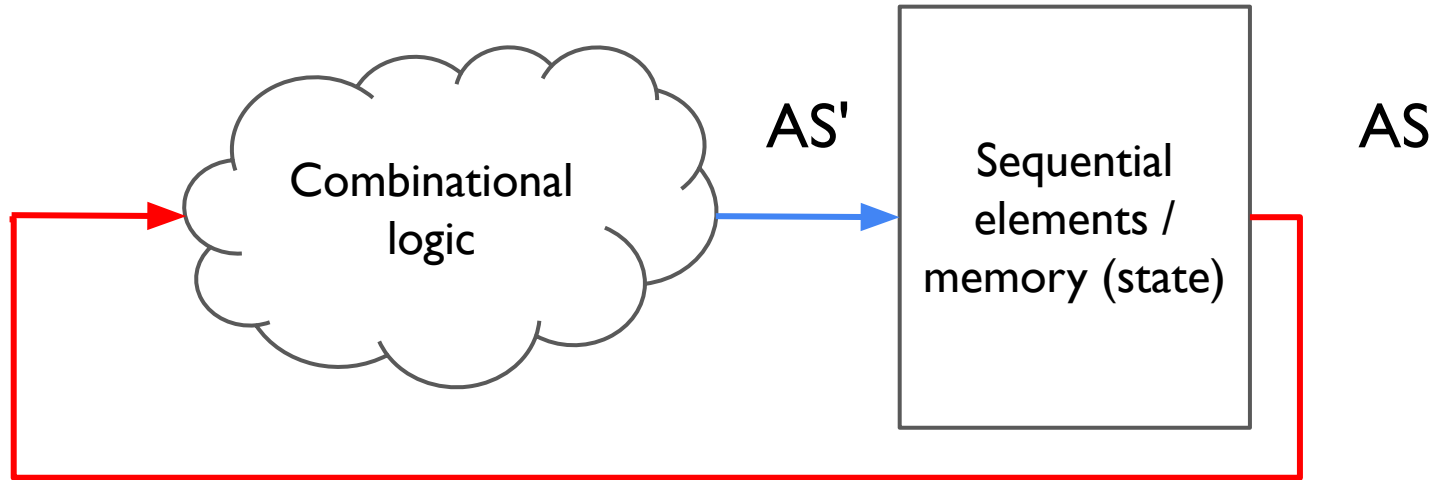
- **ISA** specifies what AS' should be given instruction and AS
- **Microarchitecture** implements how AS transforms to AS'
 - **How instruction** is processed by machine (organization)
 - Choice 1: **AS** → **AS'** (transform in 1 cycle)

Instruction processing

- **ISA** specifies what AS' should be given instruction and AS
- **Microarchitecture** implements how AS transforms to AS'
 - **How instruction** is processed by machine (organization)
 - Choice 1: $AS \rightarrow AS'$ (transform in 1 cycle)
 - Choice 2: $AS \rightarrow AS+MS1 \rightarrow AS+MS2 \rightarrow AS+MS3 \rightarrow AS'$
 - MS = microarchitectural state $\rightarrow$ programmer invisible state
 - Transform $AS \rightarrow AS'$ in multiple cycles

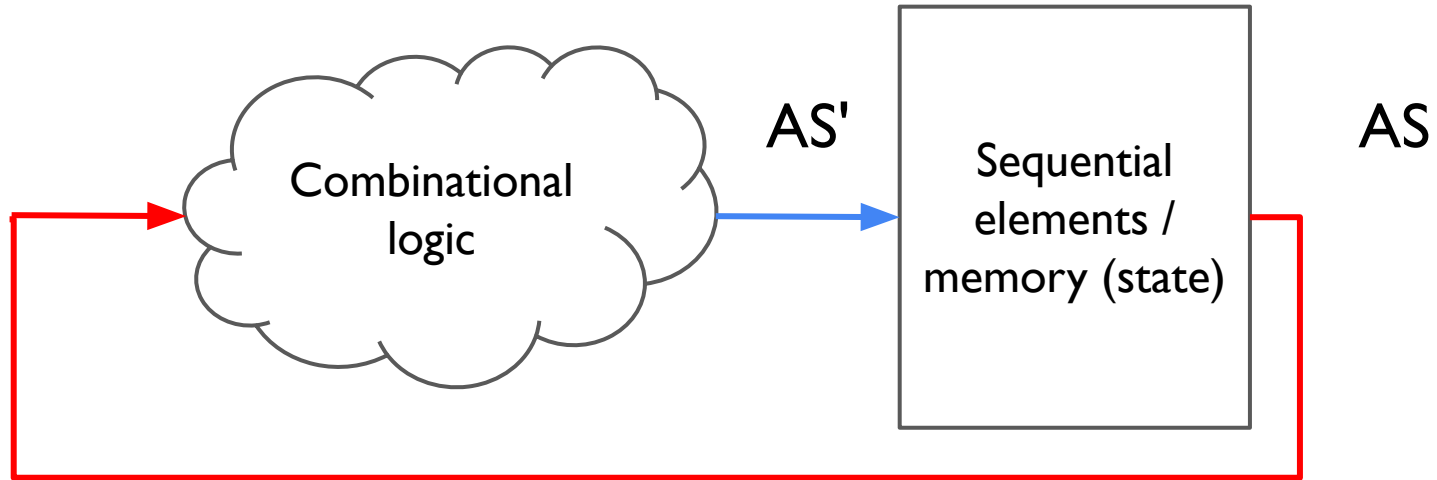
A single cycle processor

- Each instruction takes **single cycle** to execute



A single cycle processor

- Each instruction takes **single cycle** to execute

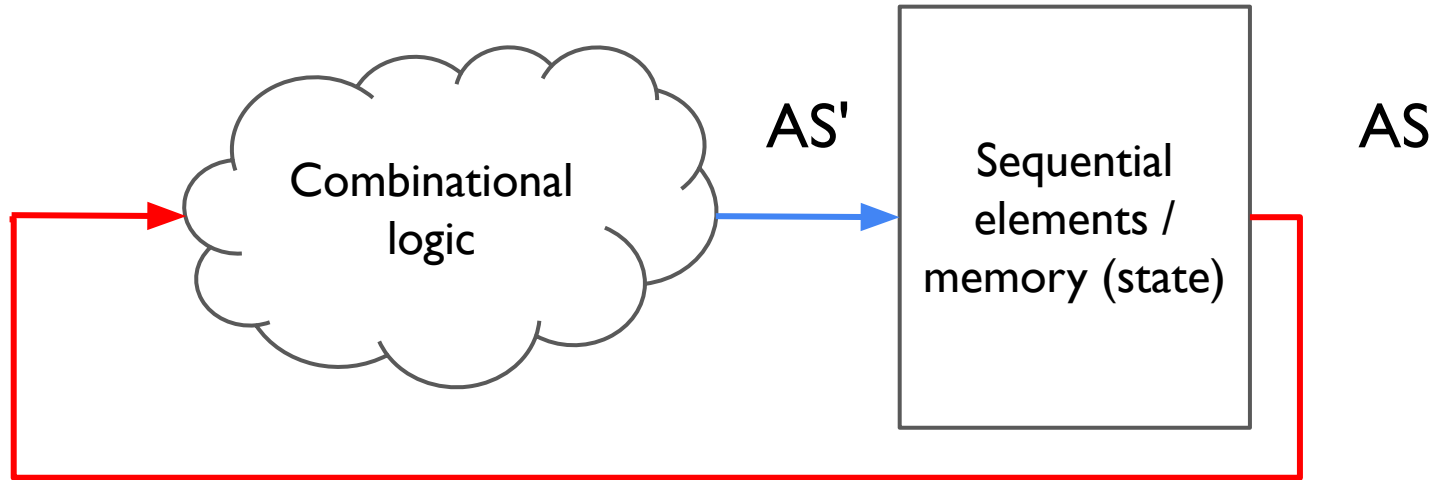


- What is **cycle time (CT)** determined by?

A single cycle processor

$$CPI = 1$$

- Each instruction takes **single cycle** to execute



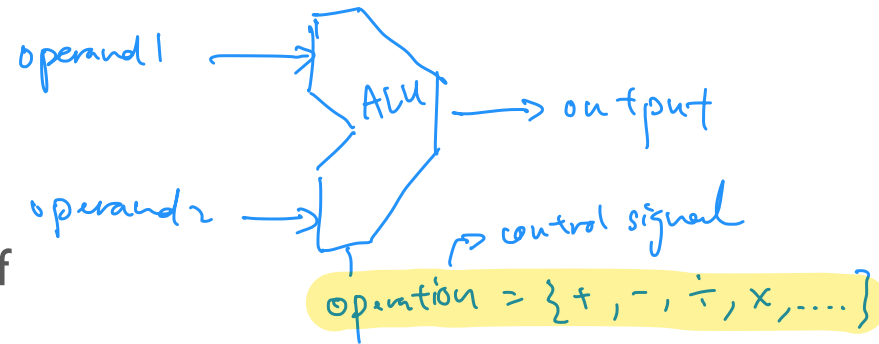
- What is **cycle time (CT)** determined by?
 - The **slowest** instruction → long cycle time

Datapath vs control logic

Instruction processing machine consists of

- **Datapath**: hardware elements that handle and transform data signals
 - functional units operate on data - ADD, AND, SUB
 - enable flow of data into functional units and registers
 - registers

Datapath vs control logic

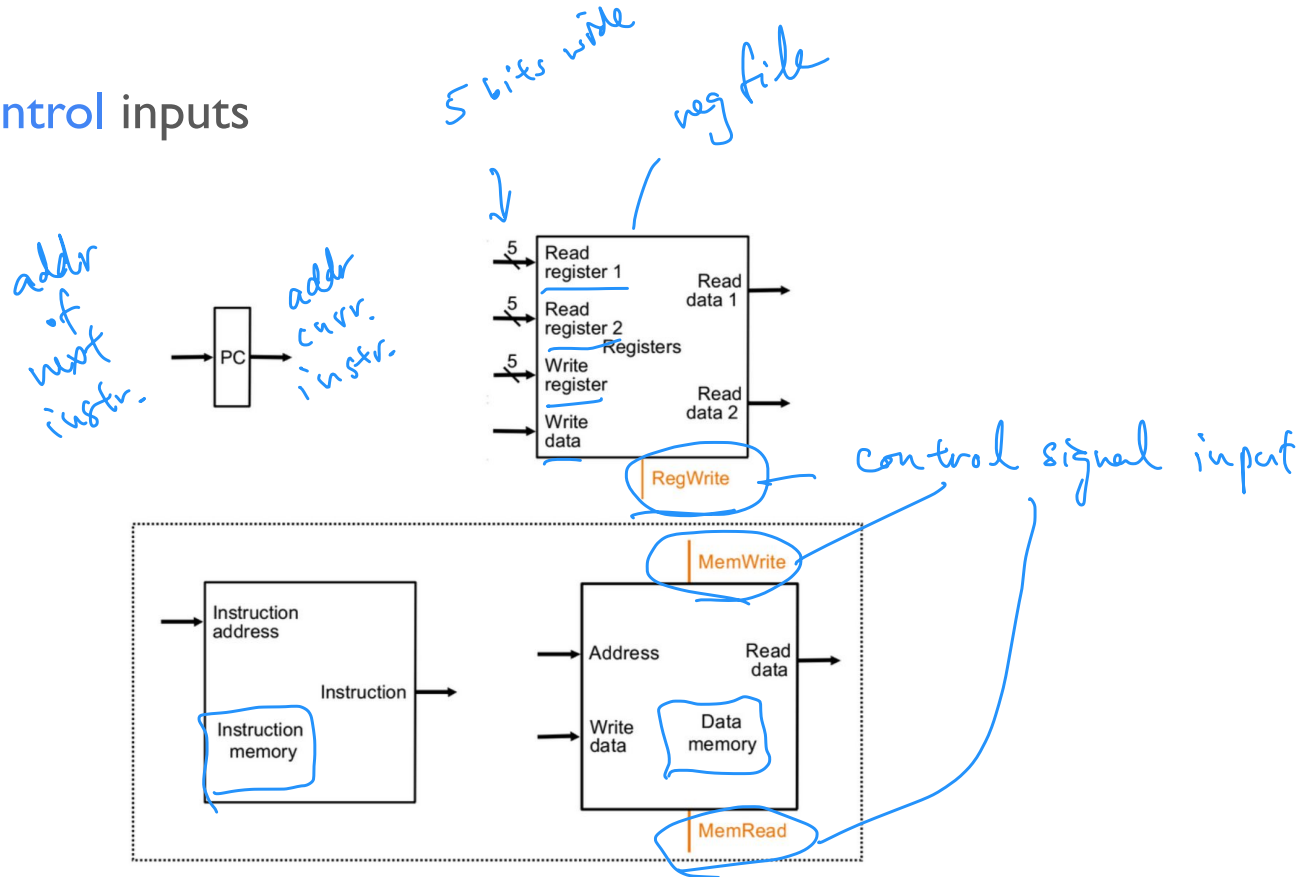


Instruction processing machine consists of

- **Datapath**: hardware elements that **handle** and **transform** data signals
 - functional units operate on data
 - enable flow of data into functional units and registers
 - registers
- **Control logic**: hardware elements that **determine signals** that **specify** what **datapath elements** should **do** to the data
 - aka **control signals**

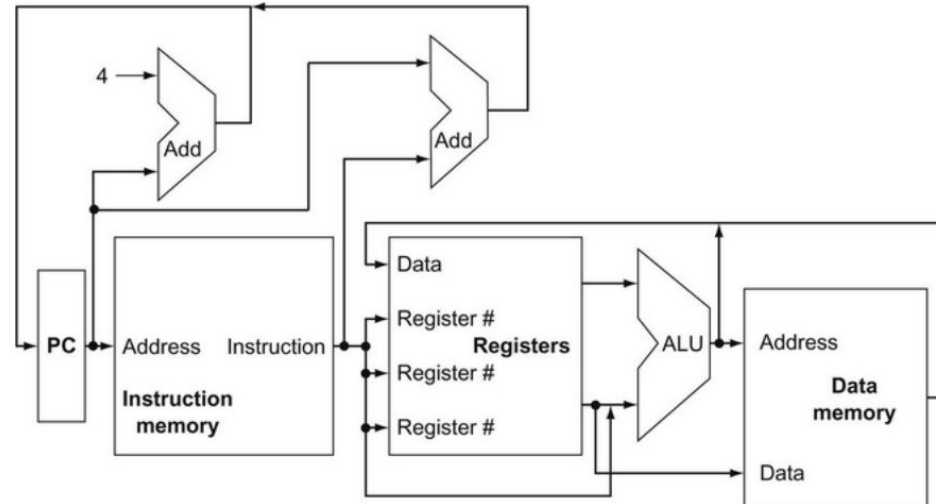
Architectural state elements

- Data and control inputs



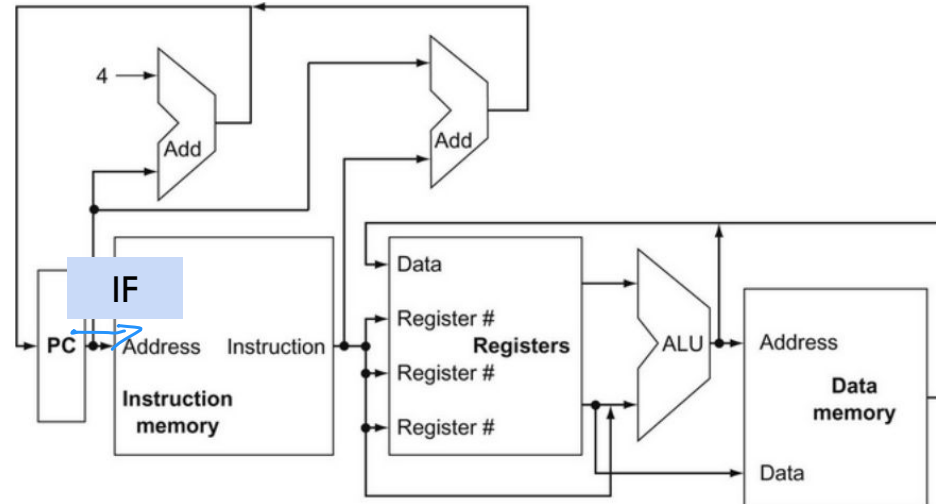
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)



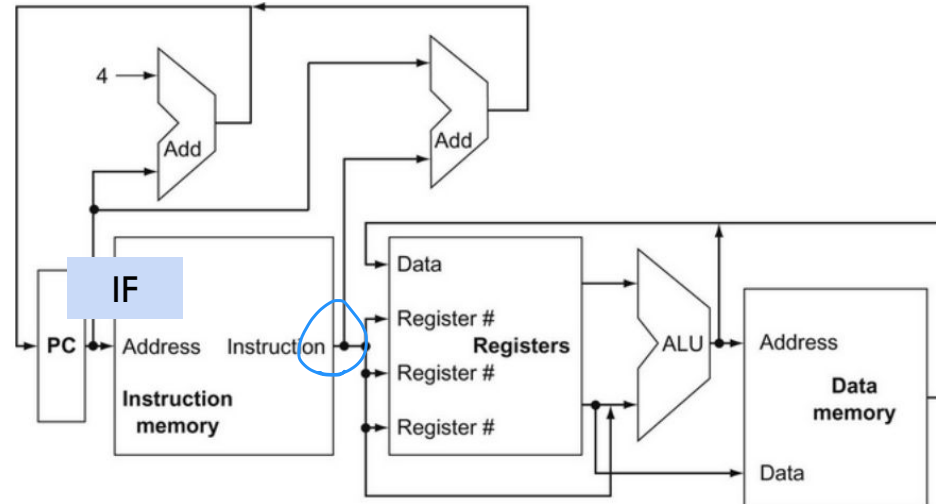
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)



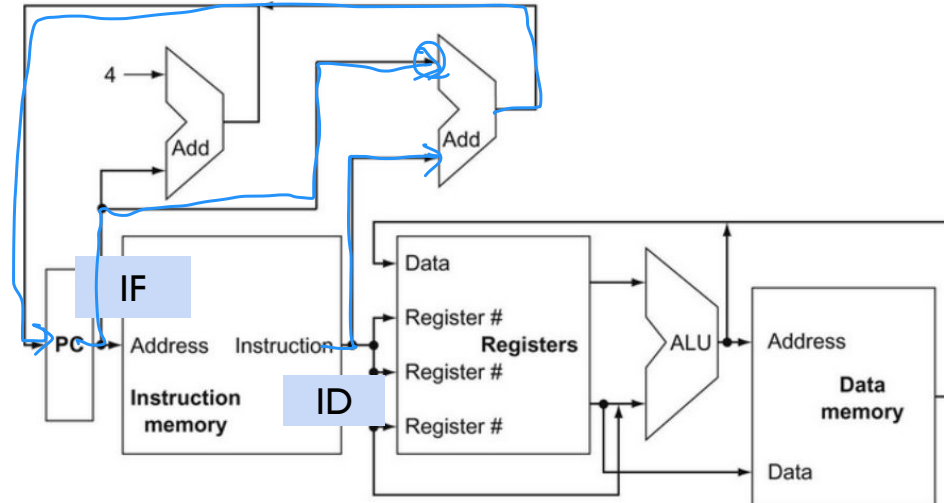
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)



Instruction processing steps

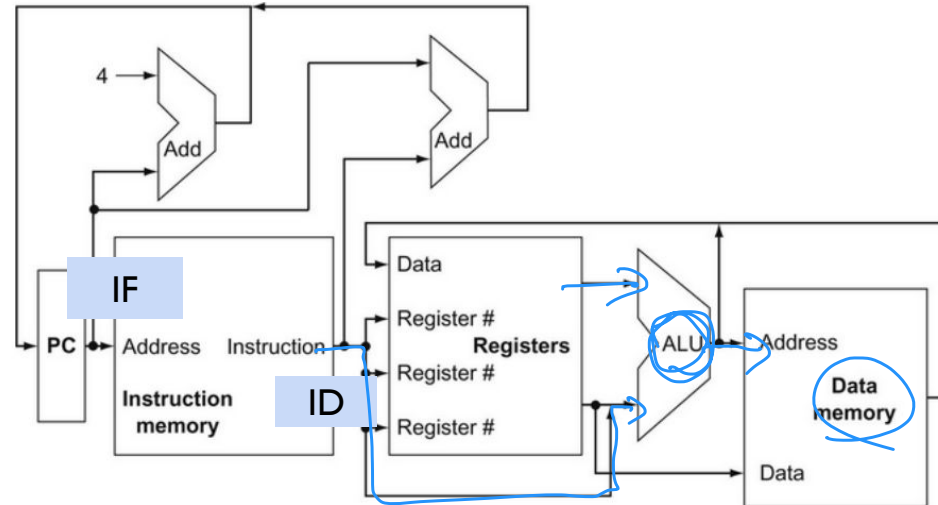
- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)



Instruction processing steps

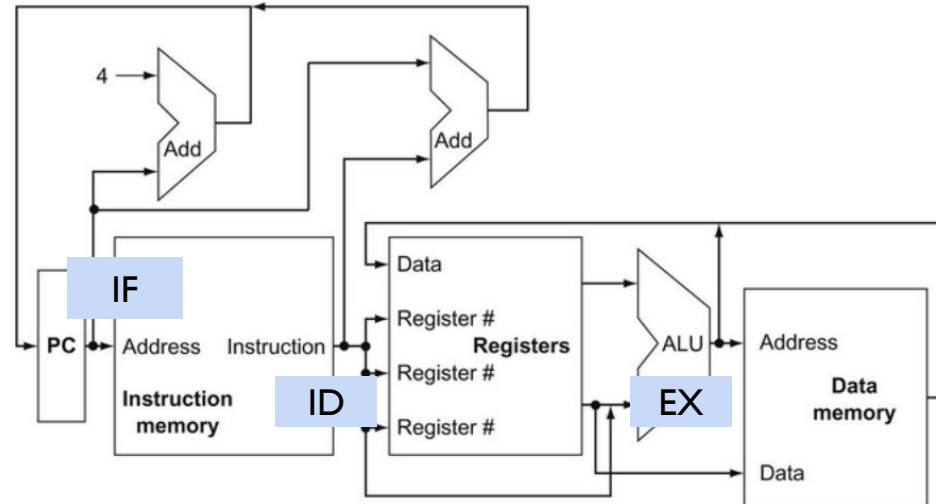
- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)
 - Execute/evaluate memory address (EX)

store x7, 8(x9)
mem. addr = x9 + 8



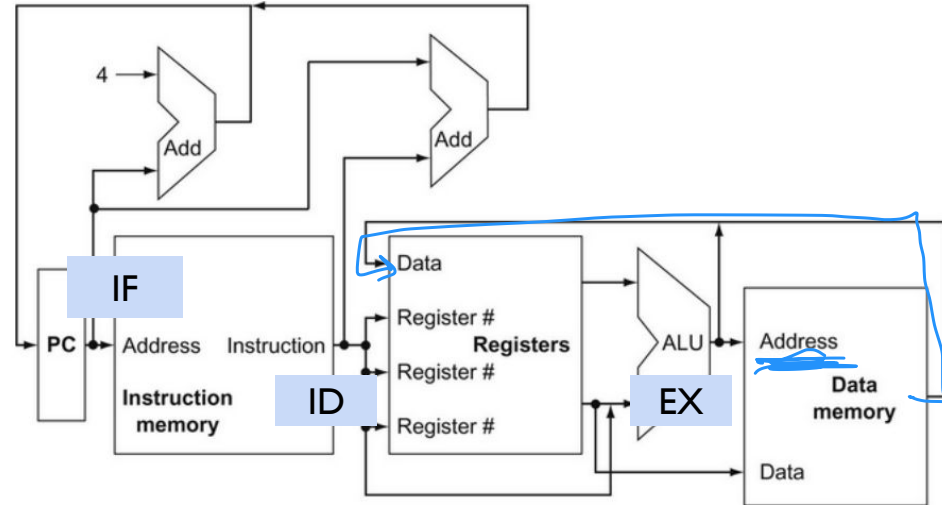
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)
 - Execute/evaluate memory address (EX)



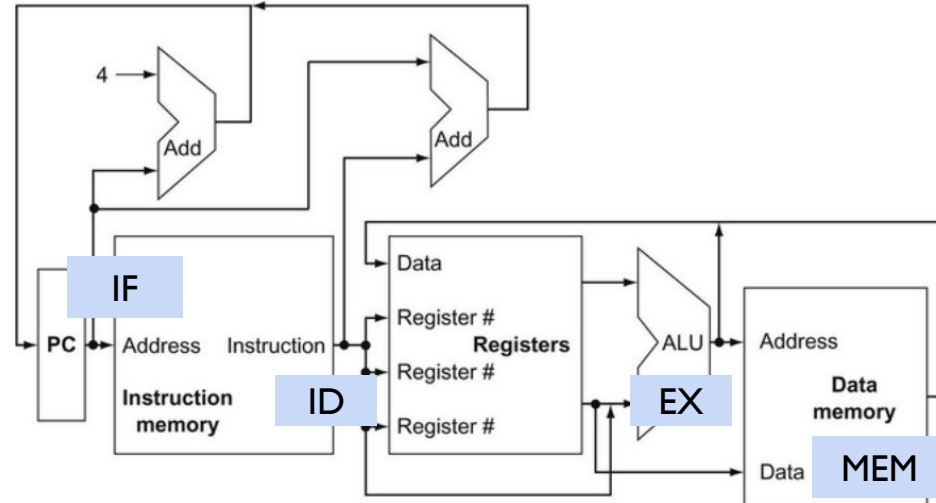
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)
 - Execute/evaluate memory address (EX)
 - Memory operand fetch (MEM)



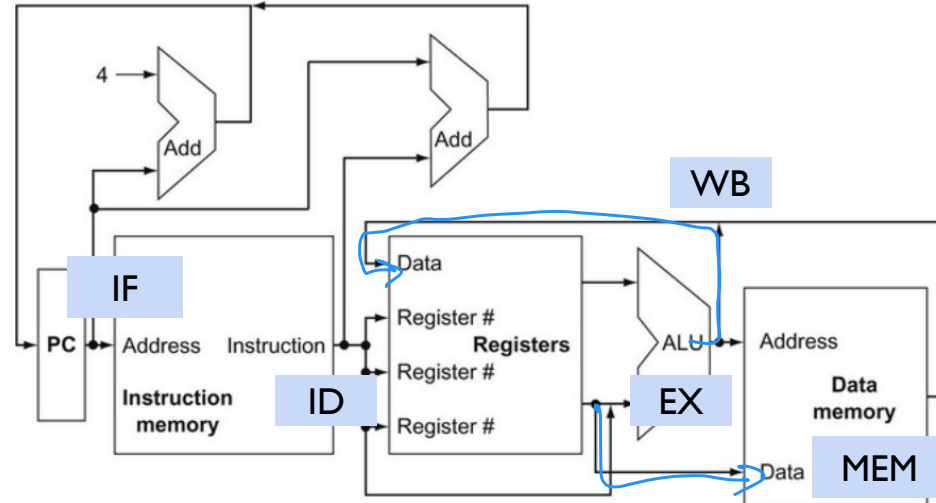
Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)
 - Execute/evaluate memory address (EX)
 - Memory operand fetch (MEM)



Instruction processing steps

- Example: 5 steps to implement RISC-V
 - Instruction fetch (IF)
 - Instruction decode and register operand fetch (ID)
 - Execute/evaluate memory address (EX)
 - Memory operand fetch (MEM)
 - Store/writeback result (WB)



Adding control signals

- Blue lines determine what datapath will be

